CMP_SC 4050

Group Algorithm Engineering Assignment 2

From Brute Force to Better Algorithms: Divide and Conquer + Dynamic Programming

Assignment type	Group Algorithm Engineering | 100 points
Assigned	Monday, Nov 1, 2026
Due	Wednesday, November 12, 2026, 11:59 PM
Topics	Divide and conquer, inversion counting, recursion, dynamic programming, memoization, coin change, validation, empirical analysis
Work mode	Group, AI-assisted with disclosure and independent validation

Assignment goal. This assignment studies two different ways to improve a correct but expensive baseline algorithm. In Part I, divide and conquer reduces inversion counting from quadratic to n log n by organizing pair comparisons around a merge step. In Part II, dynamic programming reduces repeated recursive work in coin change by identifying and reusing overlapping subproblems. You will implement, validate, benchmark, and explain both improvements.

AI Policy

AI-Assisted: Groups may use generative-AI tools for brainstorming, coding assistance, debugging, test generation, explanation, or documentation. The group is responsible for independently validating all generated content. AI output is not evidence of correctness. Include an AI Use Statement naming the tools used, describing how they contributed, and explaining how the group verified resulting code, claims, and experiments. If no generative AI was used, state that explicitly.

General Engineering Expectations

- Use one common repository or submission package containing source code, experiment scripts, data, and README instructions.
- Every optimized algorithm must be checked against an independently implemented baseline or exact solver on small instances.
- Use deterministic behavior unless randomness is part of an experiment. Record random seeds.
- Separate algorithm timing from plotting, file I/O, and result formatting as much as practical.
- Use asymptotic analysis to explain expected growth, and experiments to determine whether measurements are consistent with that analysis.
- Do not claim that finite testing proves correctness. Testing validates implementations on tested inputs; correctness arguments explain why the algorithm works generally.

Part I. Divide and Conquer: Counting Inversions

For an array A[1..n] of distinct values, an inversion is a pair of indices i < j such that A[i] > A[j]. The inversion count measures how far the array is from sorted order.

Part 1A. Implement Two Inversion Counters

Implement both algorithms below. Each should return the inversion count. The divide-and-conquer version must also return or produce the sorted array.

A. Brute-Force Pair Counting

Check every pair (i,j) with i<j and increment the count when A[i] > A[j]. Analyze the actual runtime of your implementation.

B. Divide-and-Conquer Sort-and-Count

Modify mergesort so that each recursive call returns a sorted subarray together with its inversion count. During the merge, count cross inversions in O(n) time.

Your implementation should make the central idea visible: when the next element from the right half is smaller than the next element from the left half, it forms an inversion with every remaining element in the sorted left half.

Component	Points	Full-credit expectation
Brute-force implementation	5	Correctly checks all pairs and returns the exact inversion count.
Divide-and-conquer implementation	10	Correct split, recursive counts, linear merge-and-count, and sorted output.
Correctness explanation	5	Explains left-half, right-half, and cross inversions and why each is counted exactly once.
Complexity analysis	5	Brute force Theta(n^2); divide-and-conquer recurrence and Theta(n log n) conclusion.

Part 1B. Validation and Runtime Scaling

Use the brute-force counter as the small-instance oracle for the divide-and-conquer implementation.

- Create at least 10 hand-checkable arrays, including already sorted, reverse sorted, nearly sorted, and randomly ordered examples.
- Generate at least 200 random arrays small enough for the brute-force solver.
- Require the two implementations to agree exactly on every validation array.
- Then benchmark both methods on increasing n. Include sizes large enough that their scaling behavior becomes visibly different.
- Repeat timings and report a stable representative statistic such as the median.

In the report, identify approximately where the O(n log n) method begins to outperform brute force on your machine, and explain why this crossover is implementation- and hardware-dependent.

Component	Points	Full-credit expectation
Validation design	3	Hand tests plus at least 200 randomized cross-checks.
Runtime experiment	3	Increasing n, repeated timings, fair timing methodology.
Results presentation	2	Readable table/graph comparing both algorithms.
Interpretation	2	Connects observed growth to Theta(n^2) versus Theta(n log n), including crossover discussion.

Part II. Dynamic Programming: Coin Change

You are given positive integer coin denominations D = {d1, d2, ..., dk} and a target amount A >= 0. You may use an unlimited number of coins of each denomination. Return a multiset of coins summing exactly to A with the minimum number of coins, or report IMPOSSIBLE if no solution exists.

Unless an experiment states otherwise, generated systems should include denomination 1 so all nonnegative targets are feasible.

Part 2A. Implement Four Coin-Change Solvers

A. Greedy Largest-Denomination First

Repeatedly choose the largest denomination that does not exceed the remaining amount. This is included as a comparison heuristic, not as an assumed correct solution for arbitrary coin systems.

B. Naive Exact Recursive Solver

Use the optimal-substructure recurrence that tries every possible final coin, with no memoization. This solver is intended only for small targets.

C. Top-Down DP with Memoization

Use the same recurrence, but cache each target amount the first time it is solved. Reconstruct one optimal coin multiset.

D. Bottom-Up DP

Compute optimal values for amounts 0,1,...,A in increasing order and store enough information to reconstruct one optimal solution.

All four solvers should use a common interface and return either (coin_count, chosen_coins) or IMPOSSIBLE.

Component	Points	Full-credit expectation
Greedy implementation	4	Correct largest-feasible-denomination rule and valid returned coin list.
Naive exact recursion	5	Correct exact recurrence with no memoization; handles impossible states.
Top-down memoized DP	6	Correct cache use, exact value, and reconstruction.
Bottom-up DP	6	Correct evaluation order, exact value, and reconstruction.
Complexity analysis	4	Accurate time and auxiliary-space analysis for each implementation.

Part 2B. Exact-Solver Validation + Greedy Counterexamples

Before using DP as ground truth, validate the exact implementations against one another.

- Create at least 10 hand-checkable test cases, including target 0, impossible cases, and cases with multiple optimal solutions.
- Run at least 100 randomized small instances. Whenever naive recursion is practical, require naive recursion, memoized DP, and bottom-up DP to agree on the optimal count.
- Independently verify that every returned coin list sums to A.
- Build an automated search for cases where greedy is feasible but uses more coins than the exact optimum.
- Report at least three distinct greedy counterexamples.

Required sanity check. Your counterexample search should detect D = {1,3,4}, A = 6: greedy gives 4+1+1 (3 coins), while an optimum is 3+3 (2 coins).

For at least one counterexample, explain structurally why the largest-coin local decision prevents a better global combination.

Component	Points	Full-credit expectation
Exact-solver cross-validation	4	Meaningful hand tests and at least 100 randomized small tests.
Solution validator	2	Returned sums/counts checked independently.
Automated greedy search	4	Programmatic search against exact DP.
Counterexamples and interpretation	5	Three valid examples plus a clear explanation of greedy failure.

Part 2C. Memoization and Pseudo-Polynomial Behavior

Experiment 1: Repeated Subproblems

Instrument the naive recursive and top-down memoized solvers. Measure recursive calls, distinct target states solved, and memoization cache hits. Increase A until the naive recursion becomes impractical. Use a documented safety cutoff.

Explain why the recurrence is essentially the same in both algorithms even though the amount of computation can be dramatically different.

Experiment 2: Pseudo-Polynomial Scaling

The bottom-up algorithm takes O(kA) time. Keep k fixed and small while increasing A geometrically, for example 2^8, 2^9, 2^10, and so on as far as practical.

- Record A.
- Record the bit length of A, approximately floor(log2 A)+1.
- Record the number of DP states processed.
- Record observed runtime.
- Explain why work linear in A may be exponential in the number of bits needed to encode A.

Component	Points	Full-credit expectation
Memoization instrumentation	4	Call counts, distinct states, and cache hits collected correctly.
Repeated-subproblem experiment	3	Systematic target scaling and clear results.
Pseudo-polynomial experiment	4	Geometric A growth, state/runtime measurements, reproducible setup.
Interpretation	4	Explains both overlapping subproblems and why O(kA) is pseudo-polynomial.

Part III. Cross-Paradigm Engineering Reflection

Your final report should compare the two optimization stories rather than treating them as unrelated coding exercises.

1. In inversion counting, what work does divide and conquer avoid compared with checking every pair?
2. Why are the recursive subproblems in inversion counting naturally independent before the merge, while coin-change subproblems overlap heavily?
3. What role does sorting play in the divide-and-conquer improvement?
4. The naive recursive and memoized coin-change algorithms use essentially the same recurrence. Why can their runtimes be dramatically different?
5. How did baseline/exact implementations help you validate faster algorithms in both halves of the assignment?
6. If you were given a new optimization problem, what evidence would make you try divide and conquer first? What evidence would make you try dynamic programming?

Component	Points	Full-credit expectation
All prompts addressed	3	Every question answered clearly.
Specific evidence	3	Uses results, counterexamples, or timing observations from the group's experiments.
Paradigm comparison	4	Clearly distinguishes independent recursive structure from overlapping-subproblem structure and explains when each paradigm is promising.

Required Deliverables

- Source code for inversion counters, coin-change solvers, validators, generators, counterexample search, and experiment scripts.
- README with language/version information and exact instructions for reproducing the main experiments.
- Engineering report, approximately 6-8 pages, containing methods, correctness/complexity discussion, validation, experimental results, interpretation, and reflection.
- Machine-readable results (CSV, JSON, or equivalent) underlying the main plots and tables.
- Contribution statement describing each group member's primary contributions.
- AI Use Statement describing tools used and how outputs were independently validated, or stating that no generative AI was used.

100-Point Summary

Component	Points	Full-credit expectation
Part 1A: Inversion algorithms	25	Brute force and divide-and-conquer implementations, correctness, complexity.
Part 1B: Inversion validation/runtime	10	Cross-validation and empirical scaling comparison.
Part 2A: Coin-change solvers	25	Greedy, naive recursion, memoized DP, bottom-up DP.
Part 2B: Validation/counterexamples	15	Exact-solver validation and automated greedy failure search.
Part 2C: DP experiments	15	Memoization instrumentation and pseudo-polynomial scaling.
Part III: Reflection	10	Cross-paradigm engineering analysis.